

第19回 クラスでキャラクターをまとめる

今回のゴール

- キャラクターの情報(フィールド)と動き(メソッド)を、`Character` クラスにまとめられる。
- コンストラクタを書き、`new` でインスタンスを作れる。
- 「インスタンスを入れた変数は、インスタンスの場所(参照)を持っている」ことを説明できる。

前回とのつながり

第18回までは、勇者の名前・HP・攻撃力が、`heroName`、`heroHp`、`heroAttack` のようにばらばらの変数だった。敵を 1 体増やすたびに、変数を 4 つずつ増やすことになる。今回は、キャラクター 1 人分の情報と動きを クラス にまとめる。前期に学んだクラス・インスタンス・コンストラクタを、バトルの中で使い直す回である。

実習時のフォルダ構成

```
ワークスペース
├─ java19_sample
│   └─ src
│       ├── Character.java
│       └─ Main.java
└─ java19_exercise
    └─ src
        ├── Character.java
        └─ Main.java
```

- 配布ファイル：`プログラム/第19回/sample/`、`プログラム/第19回/exercise/` (それぞれ 2 ファイル)
- 解答例：`プログラム/第19回/answer/`
- クラスは 1 ファイルに 1 つ。`src` を右クリック →「新規」→「クラス」で、`Character` と `Main` を作る(`main` メソッドのチェックは `Main` だけ)。

講義内容

1. クラスとインスタンス

用語	意味	バトルでの例
クラス	設計図	<code>Character</code> (キャラクターとは何を持ち、何ができるか)
インスタンス	設計図から作った実物	勇者、ゴブリン
フィールド	インスタンスが持つ情報	<code>name</code> 、 <code>hp</code> 、 <code>maxHp</code> 、 <code>power</code>

用語	意味	バトルでの例
メソッド	インスタンスができること	<code>showStatus()</code> 、 <code>attack(...)</code> 、 <code>heal(...)</code>

1 つのクラスから、インスタンスをいくつでも作れる。勇者とゴブリンは同じ `Character` クラスから作られ、それぞれ自分の `hp` を持つ。

2. Character クラス

プログラム/第19回/sample/Character.java

```
public class Character {
    // フィールド (キャラクターが持つ情報)
    String name;
    int hp;
    int maxHp;
    int power;

    // コンストラクタ (new したときに最初に実行される)
    Character(String name, int hp, int power) {
        this.name = name;
        this.hp = hp;
        this.maxHp = hp;
        this.power = power;
    }

    // 名前と HP を表示する
    void showStatus() {
        System.out.println(name + " HP:" + hp + "/" + maxHp);
    }

    // target に攻撃する (3 ターンごとに会心の一撃で 1.5 倍)
    void attack(Character target, int turn) {
        int damage = power;
        if (turn % 3 == 0) {
            System.out.println("会心の一撃!");
            damage = (int) (power * 1.5);
        }
        System.out.println(name + "の攻撃! " + target.name + "に" + damage + "のダメージ");
        target.hp -= damage;
    }

    // HP を回復する (最大 HP を超えない)
    void heal(int amount) {
        hp += amount;
        if (hp > maxHp) {
            hp = maxHp;
        }
    }

    // HP が残っていれば true
    boolean isAlive() {
        return hp > 0;
    }
}
```

```
}
```

- コンストラクタは、クラスと同じ名前、戻り値の型を書かない。`new` したときに 1 回だけ実行され、フィールドに最初の値を入れる。
- `this.name` は「このインスタンスのフィールド `name`」、右側の `name` は引数。名前が同じなので `this.` で区別する。
- 第18回では `static` を付けていたが、クラスのメソッドには付けない。「インスタンスが自分のフィールドを使って動く」メソッドだから。
- クラスの中のメソッドは、自分のフィールドを引数なしで使える (`showStatus()` の中の `name`、`hp`)。

3. インスタンスを作って使う

プログラム/第19回/sample/Main.java (変わった部分)

```
// キャラクターを作る (インスタンスの生成)
Character hero = new Character("勇者", 30, 8);
Character enemy = new Character("ゴブリン", 40, 7);
int herbCount = 2;
```

```

while (hero.isAlive() && enemy.isAlive()) {
    System.out.println();
    System.out.println("--- ターン" + turn + " ---");
    hero.showStatus();
    enemy.showStatus();

    int command = inputCommand(scanner, commands);

    switch (command) {
        case 1:
            hero.attack(enemy, turn);
            break;
        case 2:
            if (herbCount > 0) {
                hero.heal(15);
                herbCount--;
                System.out.println(hero.name + "は薬草を使った！ HPが" + hero.hp + "になった");
            } else {
                System.out.println("薬草がない！");
            }
            break;
        case 3:
            System.out.println(hero.name + "はにげだした！");
            escaped = true;
            break;
    }

    if (escaped || !enemy.isAlive()) {
        break;
    }

    enemy.attack(hero, turn);
    turn++;
}

```

- インスタンス名.メソッド名(...) で、そのインスタンスに命令する。`hero.attack(enemy, turn)` は「勇者が、ゴブリンに攻撃する」。
- インスタンス名.フィールド名 で、フィールドを読む(`hero.name`、`hero.hp`)。
- `!` は「～でない」。`!enemy.isAlive()` は「敵が生きていない」。
- 敵の攻撃も `enemy.attack(hero, turn)` の 1 行になった。勇者と同じメソッドを使っている。

4. 参照:インスタンスを渡すと、元のインスタンスが変わる

第18回では、`int` の値をメソッドに渡しても、元の変数は変わらなかった。ところが `hero.attack(enemy, turn)` を実行すると、`enemy` の HP は減っている。

- `enemy` という変数には、インスタンスそのものではなく、**インスタンスがある場所(参照)**が入っている。
- メソッドの引数 `target` には、その場所がコピーされる。`target` と `enemy` は **同じインスタンス** を指すので、`target.hp -= damage;` で元のゴブリンの HP が減る。

```
Character a = new Character("勇者", 30, 8);
Character b = a;          // b は a と同じインスタンスを指す（コピーではない）
b.hp = 10;
System.out.println(a.hp); // 10
```

5. 実行する前に予想する

1、1、1、2、1、1 の順に入力する。

- 3 ターン目、ゴブリンの攻撃はどうなるか(attack メソッドを読んで考える)。

6. 実行結果

ゴブリンがあらわれた！

--- ターン1 ---

勇者 HP:30/30

ゴブリン HP:40/40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン2 ---

勇者 HP:23/30

ゴブリン HP:32/40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン3 ---

勇者 HP:16/30

ゴブリン HP:24/40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

会心の一撃！

勇者の攻撃！ ゴブリンに12のダメージ

会心の一撃！

ゴブリンの攻撃！ 勇者に10のダメージ

--- ターン4 ---

勇者 HP:6/30

ゴブリン HP:12/40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 2

勇者は薬草を使った！ HPが21になった
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン5 ---

勇者 HP:14/30

ゴブリン HP:12/40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン6 ---

勇者 HP:7/30

ゴブリン HP:4/40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

会心の一撃！

勇者の攻撃！ ゴブリンに12のダメージ

ゴブリンをたおした！

- ・ゴブリンまで「会心の一撃！」を出している。勇者とゴブリンが同じ `attack` メソッドを使っているため。「勇者だけ会心の一撃を出したい」は、第21回で解決する。

7. エラーを体験する

1. `Main.java` の `new Character("勇者", 30, 8)` を `new Character("勇者", 30)` に変えて保存する。
2. コンストラクタの引数が合わない、というエラーになることを確認する。コンストラクタもメソッドと同じく、引数の数と型が合わないと呼び出せない。
3. 元に戻る。

演習1 サンプルを実行する

`java19_sample` でサンプルを実行し、予想と結果を比べる。

演習2 Character クラスを完成させる

`java19_exercise` を作り、`プログラム/第19回/exercise/` の 2 ファイルを入れる。

演習2-1 コンストラクタを書く

- ・まず、そのまま実行する。`nullがあらわれた！` と表示され、すぐに終わってしまう理由を考える。
- ・コンストラクタの中で、引数の値をフィールドに入れる。`maxHp` には `hp` と同じ値を入れる。

ヒント:

- フィールドに何も入れないと、`String` は `null`、`int` は `0` のままになる。HP が 0 なので、`while` の条件が最初から `false` になっていた。
- サンプルの `Character.java` と同じ形になる。見ずに書けるか挑戦する。

演習2-2 「ためる」コマンドを追加する

- `Character` クラスに、攻撃力を `amount` だけ上げる `powerUp(int amount)` メソッドを作る(戻り値なし)。
- `Main` の `commands` に "ためる" を追加し、`case 4` で `hero.powerUp(3);` を呼び出す。
- 呼び出した後で、`勇者は力をためた！ 攻撃力が11になった` のように表示する。

完成条件(1 ターン目に `4` を入力したとき):

```
--- ターン1 ---
勇者 HP:30/30
ゴブリン HP:40/40
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 4
勇者は力をためた！ 攻撃力が11になった
ゴブリンの攻撃！ 勇者に7のダメージ
```

発展(任意)

- 2 体目の敵(例:スライム HP 20、攻撃力 5)のインスタンスを作り、1 体目をたおした後もう一度バトルが始まるようにする。どこをくり返せばよいか考える。

確認問題

問1 出力は何か(`Character` クラスはサンプルと同じ)。

```
Character a = new Character("勇者", 30, 8);
Character b = a;
b.hp = 10;
System.out.println(a.hp);
```

問2 出力は何か。

```
class Item {
    String name;
    int price;
}
```

```
Item item = new Item();
System.out.println(item.name + " " + item.price);
```

問3 出力は何か。

```
class Slime {
    String name;

    Slime(String name) {
        name = name;
    }
}
```

```
Slime s = new Slime("スライム");
System.out.println(s.name);
```

■ 答え

- 問1: 10 (a と b は同じインスタンスを指している)
- 問2: null 0 (値を入れていないフィールドは、String などは null、int は 0。コンストラクタを書かないときは、引数なしのコンストラクタが自動で用意される)
- 問3: null (name = name; は引数に引数を入れているだけ。フィールドに入れるには this.name = name; と書く)

振り返り

- クラスとインスタンスの違いを、バトルの例で説明できるか。
- コンストラクタで this. を付ける理由を説明できるか。
- hero.attack(enemy, turn) でゴブリンの HP が減る理由を説明できるか。

次回へのつながり

今の Character は、Main から hero.hp = 999; のように HP を自由に書き換えられてしまう。HP がマイナスになったり、最大 HP を超えたりする問題も残っている。次回は、HP を安全に扱うしくみを作る。